

Klar. **Astra** soll ein langfristig angelegter, moderner **Astrofotografie-Sequencer** werden — klar in der Kategorie N.I.N.A./ASIAIR, aber mit stärkerem Fokus auf Multi-Cam, Automatisierung, moderne UX und offene Integrationen.

Produktidee

Astra ist **kein SkyHub / Observatory-OS**, sondern primär:

Ein moderner Astro-Sequencer und Imaging Controller, der technische Routine möglichst automatisch misst, kalibriert und koordiniert.

Kernprinzip:

Auto-first, aber nie Auto-only.

Wo Astra etwas zuverlässig selbst bestimmen kann, soll es das tun. Der Nutzer behält aber immer klassische **Manual-/Disabled-/Fallback-Optionen**.

Hauptziele

Astra soll langfristig bieten:

- Sequencing
- Multi-Camera / Multi-Rig
- Autofocus
- Plate Solving
- Framing
- Meridian Flip
- Guiding-Integration
- Dithering
- Polar Alignment
- Lucky Imaging
- Calibration Frames
- automatische Recovery
- moderne Integrationen wie MQTT, Home Assistant, REST, WebSocket und Webhooks
- Plugins / externe Backends

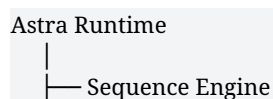
Architektur

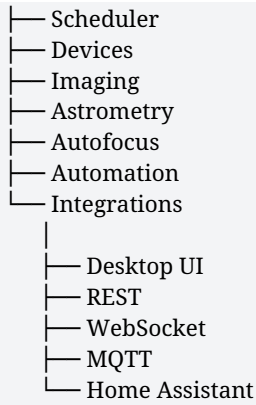
Technisch soll Astra auf **C# / .NET 10** basieren.

Für Desktop:

Avalonia als derzeit favorisierte UI-Technologie.

Die eigentliche Runtime soll aber **headless-fähig** sein.





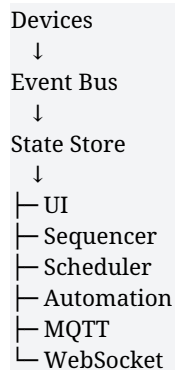
Die Desktop-UI ist nur ein Client der eigentlichen Engine.

Vorteil:

Wenn die UI crasht, läuft die Imaging-Session weiter.

Event-/State-System

Astra soll stark event-driven aufgebaut werden.



Typische States:

camera.temperature
camera.exposing
mount.tracking
guiding.rms
sequence.running
focuser.position
weather.safe

Grundidee:

Everything relevant is state or event.

Devices

Keine globale einzelne Camera.

Stattdessen ein Device Registry:

```
Devices
├── camera.main
├── camera.wide
├── mount.eq6
├── focuser.main
├── focuser.wide
├── filterwheel.main
└── guider.main
```

Devices melden außerdem **Capabilities**.

Beispiel Kamera:

```
Exposure
Cooling
Gain
Offset
ROI
VideoStreaming
HardwareTrigger
```

Das UI zeigt nur Funktionen, die das jeweilige Gerät tatsächlich unterstützt.

ASCOM / Alpaca

Astra selbst läuft auf .NET 10.

Klassische Windows-ASCOM-Treiber können über einen Compatibility/Interop-Layer angebunden werden.

Bevorzugte moderne Richtung:

ASCOM Alpaca first

```
Astra
├── Alpaca
└── ASCOM COM compatibility
```

Später eventuell native Hersteller-SDKs.

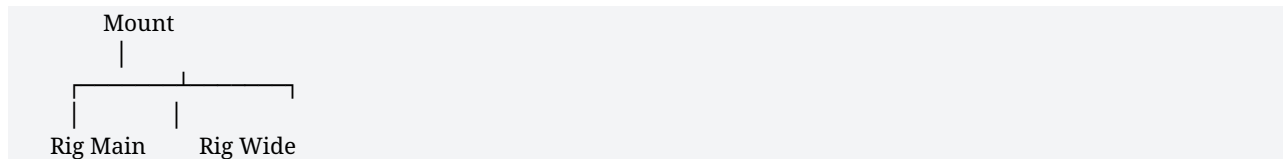
Rig-Konzept

Astra verwaltet nicht nur einzelne Devices, sondern **Rigs / Optical Trains**.

```
Rig Main
├── Camera
├── Focuser
├── Filterwheel
└── Optics

Rig Wide
├── Camera
├── Focuser
└── Optics
```

Mehrere Rigs können Ressourcen teilen:



Multi-Camera als First-Class Feature

Eines der wichtigsten Alleinstellungsmerkmale.

Nicht mehrere N.I.N.A.-Instanzen.

Eine Astra-Session kann mehrere Kameras gleichzeitig kontrollieren.

Beispiel:

```
Main Camera  
300s Ha
```

```
Wide Camera  
60s RGB
```

Während Main einmal 300 Sekunden belichtet, kann Wide fünf 60-Sekunden-Frames aufnehmen.

Astra koordiniert alles gemeinsam.

Scheduler + Resource Manager

Das ist eine der wichtigsten Kernkomponenten.

Eine Action deklariert, welche Ressourcen sie benötigt.

Beispiel:

```
Exposure Main  
requires:  
camera.main
```

```
Exposure Wide  
requires:  
camera.wide
```

→ darf parallel laufen.

Meridian Flip:

```
requires:  
mount  
camera.main  
camera.wide  
guiding
```

→ wartet auf Safe Points aller betroffenen Rigs.

Damit verhindert Astra Race Conditions und gefährliche Aktionen.

Sequencer

Der Sequencer soll nicht nur ein stumpfer linearer Tree sein.

Grundelemente:

Sequence
Parallel
Repeat
Until
Condition
Action
Trigger

Beispiel:

```
Target M31
├── Parallel
│   ├── Main Rig
│   │   └── 40 × Ha 300s
│   └── Wide Rig
│       └── 120 × RGB 60s
└── Finish
```

Pause, Resume und Abort müssen sauber und recoverbar sein.

Adaptive Sequencing

Langfristig soll Astra nicht zwingend stumpf:

A → B → C

abarbeiten.

Optional kann Astra anhand von:

- Höhe
- Meridian
- verbleibender Dunkelheit
- Mondabstand
- Filter
- bereits gesammelter Integrationszeit
- Wetter
- Priorität

entscheiden, welcher Imaging-Block als nächstes am sinnvollsten ist.

Aber klassische feste Sequenzen bleiben selbstverständlich möglich.

Multi-Cam Dithering

Astra muss Dithering über mehrere Kameras koordinieren.

Beispiel:

Main: 300s
Wide: 60s

Wide möchte nach drei Frames dithern.

Astra darf nicht dithern, solange Main noch belichtet.

Default:

Wait for all affected cameras to reach a safe point.

Dann:

Main finished
Wide finishes current frame
↓
Dither
↓
Guiding settle
↓
both continue

Manuelle Policies sollen trotzdem möglich sein.

Autofocus

Native Astra-Funktion.

Pipeline:

Focuser move
↓
Capture
↓
Star Detection
↓
HFR/FWHM
↓
next position
↓
Focus curve
↓
best focus

Mehrere Algorithmen können unterstützt werden:

- Parabolic fit
- Hyperbolic fit
- V-Curve / linear wings

Astra nutzt viele Sterne und Median/statistische Auswertung statt eines einzelnen Sterns.

Autofocus Trigger

Zum Beispiel:

- Sequence Start
- Filter Change
- alle X Minuten
- Temperaturänderung
- nach Meridian Flip
- HFR-Verschlechterung
- nach Reconnect

Alle weiterhin konfigurierbar.

Filter Offsets automatisch kalibrieren

Beispiel:

```
L → 18540  
R → 18572  
G → 18561  
B → 18588
```

Astra berechnet:

```
L  0  
R +32  
G +21  
B +48
```

Nicht nur einen Run, sondern mehrere.

Dann beispielsweise:

```
Ha:  
+76  
+82  
+79  
→ +79 ± 3 steps
```

Astra speichert Offset + Unsicherheit.

Manual Override bleibt erhalten.

Predictive Focus

Langfristig soll Astra aus historischen Daten lernen:

```
Temperature  
Filter  
Focus Position  
Altitude  
Time
```

Beispiel:

L filter:
-32 focuser steps / °C

Dann kann Astra kleine Temperaturänderungen korrigieren, ohne jedes Mal einen kompletten Autofocus-Run auszuführen.

Keine Blackbox-KI nötig; zunächst sauber statistisch/modellbasiert.

Focuser Backlash automatisch messen

Der Nutzer soll im Idealfall **keinen Backlash-Wert manuell einstellen müssen**.

Astra macht bidirektionale Fokus-Sweeps.

OUTWARD focus curve
vs.
INWARD focus curve

Backlash zeigt sich als **horizontaler Versatz der Fokuskurven**.

Beispiel:

Outward best focus: 18540
Inward best focus: 18612
→ Backlash $\approx$ 72 steps

Wichtig wegen Seeing:

Nicht aus einem einzelnen HFR-Wert ableiten.

Stattdessen:

- mehrere Frames
- viele Sterne
- Median
- mehrere Sweep-Zyklen
- komplette Kurven gegeneinander fitten

Dann:

Backlash: 73 ± 6 steps
Confidence: High

Bei schlechter Messung:

Calibration inconclusive
Using previous value: 71

Auto / Manual / Disabled

Das soll ein konsistentes Astra-Muster werden.

Beispiel:

Backlash

- Automatic
- Manual
- Disabled

Dasselbe bei:

Focus Step Size
Filter Offsets
Settling
Plate Solver
Calibration
etc.

Und Auto zeigt immer transparent, welchen Wert Astra tatsächlich verwendet.

Confidence-System

Astra soll möglichst nicht so tun, als seien Messungen perfekt.

Statt:

Backlash = 73

eher:

Backlash
 73 ± 6 steps
Confidence: High

Dieses Prinzip soll sich durch automatische Kalibrierungen ziehen.

Automatische Focus Step Size

Astra kann selbst herausfinden, wie groß Focuser-Schritte für einen guten Autofocus-Run sein sollten.

Kleine Testbewegungen:

Position
↓
HFR response
↓
choose step size

Manual bleibt möglich.

Auto Exposure für Autofocus / Plate Solve

Astra soll pro Rig herausfinden können, welche kurzen Belichtungen geeignet sind.

Ziel:

- genug Sterne
- wenig Saturation
- möglichst schnell

Diese Werte können pro Rig gespeichert werden.

Plate Solving

Langfristig nativ.

Pipeline:

```

FITS
↓
Star Detection
↓
Pattern Generation
↓
Pattern Index Lookup
↓
Candidate Verification
↓
WCS Fit
↓
RA / Dec / Rotation / Scale

```

Ergebnis:

- Bildzentrum RA/DEC
- Rotation
- Pixel Scale
- Parity
- WCS

Brennweite mandatory

Jedes Optical Train bekommt zwingend:

```

Focal Length
Sensor Width
Sensor Height
Pixel Size
Resolution

```

Daraus berechnet Astra:

- FOV
- erwartete Pixel Scale
- sinnvollen Solver-Indexbereich

Aber Brennweite ist ein **starker Hint**, keine absolute Wahrheit.

Astra lässt beispielsweise $\pm 15\text{--}20\%$ Spielraum.

Nach einem Solve kann Astra die konfigurierte effektive Brennweite plausibilisieren.

Native Plate Solver Database

Gaia DR3 dient als grundlegender Sternkatalog.

Optional Hipparcos als Ergänzung für sehr helle Sterne.

Aber Astra durchsucht nicht Gaia mit 1,8 Milliarden Quellen direkt.

Stattdessen:

```
Gaia DR3
↓
Magnitude-/Quality-Filter
↓
räumliche Auswahl
↓
Pattern Generation
↓
Astra Pattern Index
```

Patterns z. B. 4-Star Quads.

Diese bekommen skalierungs-/rotationsinvariante Fingerprints.

Pattern Index

Mehrere Scale-Packs:

```
0.2° – 0.5°
0.5° – 1°
1° – 2°
2° – 5°
5° – 15°
15° – 40°
```

Durch mandatory Brennweite kennt Astra ungefähr das FOV und lädt normalerweise nur die relevanten Scale-Indizes.

Der Index soll eher ein schnelles binäres/memory-mapped Format sein als SQLite.

Hinted vs Blind Solve

Normalfall:

```
Mount position
+
FOV
+
Pixel Scale
```

→ sehr schneller Hinted Solve.

Optional:

Blind / lost-in-space solve

wenn Position unbekannt ist.

Solver Backends

Astra soll nicht vom nativen Solver abhängig sein.

Plate Solver

- Astra Native
- ASTAP
- Astrometry.net
- ggf. PlateSolve3

Für frühe Versionen ist ASTAP wahrscheinlich der pragmatische Standard.

Native Astra Astrometry kann parallel wachsen.

Framing / Recentering

Mit Plate Solving:

```
GoTo
↓
Exposure
↓
Solve
↓
Compare actual vs target
↓
Mount correction
↓
Solve again
```

bis Target ausreichend genau zentriert ist.

Three-Point Polar Alignment

Soll native Astra-Funktion werden.

Prinzip:

```
Position 1 → solve
↓
RA move
↓
Position 2 → solve
↓
RA move
↓
Position 3 → solve
```

↓
polar error calculation

Danach Live-Anzeige während der mechanischen Einstellung:

Altitude error
Azimuth error
Total error

und kontinuierliche Updates beim Nachjustieren.

Imaging Pipeline

Viele Features sollen dieselbe zentrale Analyse-Pipeline verwenden.

Astra.Imaging.Analysis

Star Detection
HFR
FWHM
SNR
Shape
Background

Wird verwendet von:

Autofocus
Plate Solving
Lucky Imaging
Frame Quality
Tilt Analysis

Keine fünf unterschiedlichen Star-Detection-Implementierungen.

Lucky Imaging

Native Funktion.

Eigene schnelle Pipeline:

Camera stream
↓
Frame buffer
↓
Quality analysis
↓
Reject / Keep
↓
Alignment
↓
optional Live Stack

Metrics:

- FWHM
- HFR

- Sharpness
- Contrast
- Star Count
- Background
- SNR

Beispiel:

```
120 FPS
Keep best 15 %
```

Für Performance:

- pooled memory
- Memory<T>
- Span<T>
- ArrayPool<T>
- möglichst keine unnötigen Copies

Falls erforderlich später native Processing-Komponenten in Rust/C++.

Frame Quality

Jedes Sub kann automatisch Metriken bekommen:

```
HFR
FWHM
Eccentricity
Star Count
Background
Guiding RMS
```

Astra kann Ausreißer markieren.

Später kann es Ursachen unterscheiden:

```
Cloud
Guiding
Focus
Wind
Tracking
```

und darauf reagieren.

Smart Settling

Nicht pauschal:

Wait 10 seconds.

Sondern Astra beobachtet Guiding/Star Motion.

```
Dither
↓
wait until RMS/stability threshold reached
```

↓
continue

Manual Fixed-Time bleibt als Alternative.

Meridian Flip

Astra koordiniert:

```
finish exposures
↓
pause rigs
↓
stop guiding
↓
flip
↓
plate solve
↓
recenter
↓
restart guiding
↓
optional autofocus
↓
continue
```

Mit Multi-Cam als gemeinsamer koordinierter Vorgang.

Guiding

Zunächst wahrscheinlich:

PHD2 Integration

Später eventuell native Guiding Engine.

Astra kann:

- Guiding RMS beobachten
 - Settling bestimmen
 - Dither koordinieren
 - Guiding-Ausfälle recovern
-

Calibration Frames

Astra soll Flats automatisch auf Zielsignal regeln.

Beispiel:

Target ADU
↓

```
test exposure
↓
adjust duration/brightness
↓
capture flat set
```

Darks, Flat Darks und passende Master-Zuordnung können ebenfalls automatisiert werden.

Rotator Calibration

Plate Solves liefern tatsächliche Rotation.

Dadurch kann Astra:

```
Commanded: 45°
Solved: 43.7°
```

Offset automatisch bestimmen.

Manual Override bleibt.

Tilt / Field Diagnostics

Langfristig:

HFR/FWHM über das gesamte Sensorfeld auswerten.

Damit kann Astra Hinweise geben auf:

- Sensor Tilt
- Backfocus
- Feldkrümmung
- schlechte Ecke

Nicht zwingend direkt korrigieren, aber sauber diagnostizieren.

Recovery

Sehr wichtig für unattended Imaging.

Astra soll nicht nur:

```
Success / Failure
```

kennen.

Sondern Recovery Policies:

```
Retry
Reconnect
Skip
Pause
Abort Rig
```

Abort Session
Fallback

Beispiel:

Camera disconnected

Reconnect attempt 1/3

✓ Connected

Cooling restored

Sequence resumed

Session Recovery

Astra speichert regelmäßig Session-State.

Nach Neustart:

Target
Sequence position
completed frames
device state
remaining work

und kann möglichst sicher fortsetzen.

Integrationen

Wichtig, aber nicht Produktzentrum.

Geplant:

MQTT
Home Assistant
REST API
WebSocket
Webhooks

MQTT beispielsweise:

astra/sequence/state
astra/camera/main/temperature
astra/mount/state

Commands:

start
pause
resume
abort
park

HA kann über MQTT Discovery automatisch Entities bekommen.

Umgekehrt kann Astra HA-Entities als Input nutzen.

Plugin-System

Langfristig:

```
Astra.Plugin.ASTAP
Astra.Plugin.PHD2
Astra.Plugin.ZWO
Astra.Plugin.MQTT
```

Die Plugin Contracts sollen früh durchdacht werden, auch wenn dynamisches Plugin Loading erst später kommt.

UI / UX

Kein klassisches Astro-Software-Gefühl mit 30 Tabs und Tabellen.

Eher:

- flat
- schnell
- responsive
- wenige Borders
- große klare Typografie
- Dark Charcoal
- Off-White
- dezente Violet/Blue Akzente
- Status direkt sichtbar

Multi-Cam etwa:

MAIN	WIDE
Ha · 300s	RGB · 60s
 74%	 43%
-10.0°C	-10.1°C
18 / 40	52 / 120

Sequencer UX

Der Nutzer soll nicht permanent komplexe Container-Trees sehen müssen.

Mehr Timeline-/Workflow-orientiert:

- ✓ Cool camera
- ✓ Slew
- ✓ Center
- ✓ Autofocus
- Ha exposure

- Dither
- Ha exposure

Advanced Mode darf darunter weiterhin den vollständigen Tree zeigen.

Produktprinzipien

Die bisher wichtigsten:

1.

Auto-first, but never auto-only.

2.

Automate what can be measured.

3.

Automation must not remove control.

4.

Expose confidence, not fake certainty.

5.

Multi-camera is a first-class concept, not a workaround.

6.

The sequencer owns the imaging session; integrations remain integrations.

Positionierung

Nicht:

N.I.N.A. clone

sondern ungefähr:

Astra is a modern astrophotography sequencer built around multi-camera imaging, automation and a fast UX.

Die klare Kategorie bleibt **Astro Sequencer / Imaging Controller**.

Hype / Community

Astra soll möglichst **öffentlich entwickelt** werden.

Empfohlene Kanäle:

- GitHub
- Discord
- YouTube / Devlogs
- kleine Website

Nicht auf zehn Social-Plattformen gleichzeitig.

Der Entwicklungsprozess selbst erzeugt Content.

Videoideen:

Building multi-camera astrophotography sequencing

How autofocus actually works

Automatically measuring focuser backlash

Building a native plate solver in C#

Why dithering is hard with multiple cameras

Automatic filter offset calibration

Das passt gleichzeitig zu Astro und Software-Development.

Erstes großes Showcase-Feature

Multi-Camera Sequencing.

Message:

3 cameras. One session.

Das ist sofort verständlich und unterscheidet Astra sichtbar.

Danach gute Showcase-Features:

- Auto Filter Offsets
- Auto Backlash
- Smart Dithering
- Autofocus
- Plate Solve / Framing

Releases

Nicht bis „1.0“ warten.

Eher:

0.0.x Simulator / architecture

0.1 Single-camera sequencing

0.2 Multi-camera

0.3 Autofocus
0.4 ASTAP / plate solving
0.5 Polar alignment
...

Immer klar als Alpha/Beta kennzeichnen.

Roadmap grob

Phase 0 — Foundation

- Repo
- Architektur-Dokument
- Branding
- .NET 10 Solution
- Avalonia Shell
- Logging
- DI
- Config
- Event Bus
- State Store
- Simulator Devices

Phase 1 — Runtime

- Device Registry
- Rig Registry
- Headless Runtime
- REST/WebSocket
- Resource Manager
- Scheduler

Phase 2 — Hardware

- Alpaca
- ASCOM compatibility
- Camera
- Mount
- Focuser
- Filter Wheel
- Multi-Cam

Phase 3 — Sequencing

- Sequential
- Parallel
- Repeat

- Conditions
- Safe Points
- Pause/Resume
- Recovery

Phase 4 — Imaging

- FITS
- preview
- histogram
- Star Detection
- HFR/FWHM
- Frame Metrics

Phase 5 — Autofocus

- native autofocus
- automatic step size
- filter offset calibration
- automatic backlash
- temperature compensation

Phase 6 — Astrometry

- ASTAP first
- solving
- recenter
- framing
- WCS
- three-point polar alignment
- native solver research

Phase 7 — Advanced

- Lucky Imaging
 - adaptive scheduling
 - calibration automation
 - tilt analysis
 - predictive focus
 - deeper recovery
 - plugin SDK
-

Wenn ich Astra gerade auf einen Satz reduzieren müsste:

Astra wird ein moderner .NET-Astrofotografie-Sequencer, der Multi-Camera-Imaging nativ beherrscht und möglichst viele technische Parameter automatisch misst und kalibriert, ohne erfahrenen Nutzern die manuelle Kontrolle wegzunehmen.

Das ist inzwischen ziemlich klar und als Langzeitprojekt groß genug, dass du daran wirklich Jahre sinnvoll weiterbauen kannst.